🔌 CATEGORY 2: Providers

> **Difficulty:** Beginner → Intermediate | **Topics:** 5 | **Terraform Interview Mastery Series**

Table of Contents

1. What Providers Are and How They Work Internally
2. Provider Configuration and Authentication Patterns
3. Provider Versioning and `required_providers`
4. ⚠ Multiple Provider Instances — Aliases and Cross-Region/Account
5. Provider Caching and Plugin Mirror Strategies

Topic 7: What Providers Are and How They Work Internally

🔵 What It Is (Simple Terms)

A **provider** is a plugin that teaches Terraform how to talk to a specific API. Without providers, Terraform Core has no idea what an EC2 instance or an S3 bucket or a DNS record is. Providers are the bridge between your HCL configuration and the real-world API of AWS, GCP, Azure, Cloudflare, Datadog, GitHub, or any other platform.

Think of Terraform Core as a general-purpose engine and providers as **adapters** — each one purpose-built for a specific platform.

🔵 Why It Exists — What Problem It Solves

If Terraform Core knew about every cloud API directly:

- The Terraform binary would be enormous
- Adding support for a new cloud would require changing Core
- Third-party platforms (Datadog, PagerDuty, Snowflake) could never be supported
- Every API change in AWS would require a Terraform Core release

The provider plugin architecture solves this by:

- **Decoupling** cloud-specific logic from the orchestration engine
- Letting **HashiCorp, cloud vendors, and the community** maintain their own providers independently
- Allowing **Terraform Core to manage any API** that has a provider — even internal ones

---

🔵 How Providers Work Internally

The Plugin Protocol

Terraform Core and providers communicate via **gRPC** using the **Terraform Plugin Protocol**. Providers run as **separate OS processes** — not embedded in the Terraform binary.

```
┌──────────────────────────────────────────────────────────────────────────┐
│                     Provider Execution Model                             │
│                                                                          │
│  terraform apply                                                         │
│       │                                                                  │
│       ▼                                                                  │
│  Terraform Core                                                          │
│       │ starts subprocess                                                │
│       ▼                                                                  │
│  .terraform/providers/registry.terraform.io/hashicorp/aws/5.1.0/    │
│       linux_amd64/terraform-provider-aws_v5.1.0_x5                   │
│       │                                                                  │
│       │ gRPC over local socket                                           │
│       ▼                                                                  │
│  ┌──────────────────────────────────────────────┐                      │
│  │            Provider Process                  │                      │
│  │                                              │                      │
│  │  GetSchema()     → Returns resource schemas │                      │
│  │  PlanResourceChange() → Computes diff       │                      │
│  │  ApplyResourceChange() → Makes API calls    │                      │
│  │  ReadResource()  → Refreshes state          │                      │
│  │  ImportResourceState() → Imports existing   │                      │
│  └──────────────────────────────────────────────┘                      │
│                      │ HTTPS API calls                                   │
│                      ▼                                                   │
│             AWS / GCP / Azure APIs                                       │
└──────────────────────────────────────────────────────────────────────────┘
```

The Provider Lifecycle During a `terraform apply`

```
Step 1: terraform init
  → Downloads provider binary from registry.terraform.io
  → Stores in .terraform/providers/<source>/<version>/<os_arch>/
  → Records version + checksum in .terraform.lock.hcl

Step 2: terraform plan/apply starts
  → Core forks provider binary as subprocess
  → Establishes gRPC connection over Unix socket (or named pipe on
Windows)
  → Calls GetProviderSchema() → provider returns all resource/data source
schemas

Step 3: Provider configuration
  → Core sends ConfigureProvider() call with provider block contents
(region, credentials)
```

```
    → Provider validates config, sets up SDK client (e.g. AWS SDK Go v2)

  Step 4: Plan phase
    → Core calls PlanResourceChange() for each changed resource
    → Provider validates config, calculates expected diff
    → Returns proposed new state to Core

  Step 5: Apply phase
    → Core calls ApplyResourceChange() for each resource in dependency order
    → Provider executes API calls (CreateInstance, UpdateBucket,
  DeleteSecurityGroup)
    → Returns new resource state (including computed attributes like IDs,
  ARNs)
    → Core writes state to backend

  Step 6: Core terminates provider subprocess
```

What a Provider Defines

Each provider exposes:

```
  Resources       →  Infrastructure objects you can create/update/destroy
                     aws_instance, aws_s3_bucket, aws_security_group

  Data Sources    →  Read-only queries against existing infrastructure
                     data.aws_ami, data.aws_vpc, data.aws_caller_identity

  Functions       →  Built-in functions (newer providers, Terraform 1.8+)
                     provider::aws::arn_parse()
```

🔵 Provider SDK Versions

There are two SDKs for writing providers:

SDK	Status	Used By
Plugin SDK v2	Stable, widely used	Most existing providers
Plugin Framework	New, recommended for new providers	Newer providers (AWS CC, etc.)

As an engineer using Terraform, you don't typically write providers — but understanding that providers are Go programs using these SDKs helps with debugging.

🔵 The Provider Registry

The public registry lives at `registry.terraform.io`. Providers are identified by:

```
registry.terraform.io / hashicorp / aws
└── hostname          └── namespace └── type

# Short form (hostname assumed to be registry.terraform.io):
hashicorp/aws
hashicorp/google
hashicorp/azurerm

# Community providers:
hashicorp/kubernetes
datadog/datadog
cloudflare/cloudflare
mongodb/mongodbatlas
```

Provider tiers:

```
Official   →  Maintained by HashiCorp (hashicorp/aws, hashicorp/google)
Partner    →  Maintained by technology partners (datadog/datadog)
Community  →  Maintained by community (individual contributors)
```

🔵 Short Interview Answer

> "Providers are Terraform plugins that bridge the gap between Terraform Core and real-world APIs. They run as separate OS processes and communicate with Core via gRPC using the Terraform Plugin Protocol. Each provider defines the resources and data sources it manages, handles authentication with the target API, and translates Terraform's CRUD operations into actual API calls. They're downloaded during `terraform init` from registry.terraform.io and version-locked in `.terraform.lock.hcl`."

🔵 Deep Dive Answer

> "Internally, providers implement a defined gRPC interface. When Terraform Core starts a plan or apply, it forks the provider binary as a subprocess and establishes a gRPC connection over a local Unix socket. Core first calls `GetProviderSchema` to learn all supported resource types and their attribute schemas. Then for each resource, it calls `PlanResourceChange` during plan phase — the provider validates the config and returns a proposed new state. During apply, `ApplyResourceChange` is called — the provider actually makes the API call and returns the resulting state including computed attributes like resource IDs and ARNs. This separation means a provider crash doesn't crash Core, providers can be updated independently of Terraform, and anyone can write a provider for any API."

🔵 Real World Production Example

```
# A real production setup using multiple providers together
# Each provider is a separate binary, separate process, separate API

terraform {
  required_providers {
    aws        = { source = "hashicorp/aws",       version = "~> 5.0" }
    datadog    = { source = "datadog/datadog",     version = "~> 3.0" }
    cloudflare = { source = "cloudflare/cloudflare", version = "~> 4.0" }
    random     = { source = "hashicorp/random",    version = "~> 3.5" }
  }
}

# AWS provider — provisions EC2, RDS, S3
resource "aws_instance" "web" { ... }

# Datadog provider — creates monitors for the new instance
resource "datadog_monitor" "cpu" {
  name  = "High CPU on ${aws_instance.web.id}"
  type  = "metric alert"
  query = "avg(last_5m):avg:aws.ec2.cpuutilization{instance-
id:${aws_instance.web.id}} > 90"
}

# Cloudflare provider — creates DNS record pointing to the instance
resource "cloudflare_record" "web" {
  zone_id = var.cloudflare_zone_id
  name    = "web"
  value   = aws_instance.web.public_ip
  type    = "A"
}
```

Three separate API providers, one Terraform config, one `apply`. This is the power of the provider architecture.

🔵 Common Interview Questions

Q: What is the Terraform Plugin Protocol?

> "It's the gRPC-based interface that Terraform Core uses to communicate with provider processes. Providers must implement a set of RPC methods: GetProviderSchema, ConfigureProvider, PlanResourceChange, ApplyResourceChange, ReadResource, and ImportResourceState. Core calls these methods during plan and apply. There are two versions: the older Plugin SDK v2 and the newer Plugin Framework — both implement the same protocol but with different Go APIs for provider authors."

Q: What happens if a provider crashes during apply?

> "Since providers run as separate processes, a provider crash doesn't crash Terraform Core. Core will receive an error from the gRPC call and report it as an apply error. Resources that were being managed by the crashed provider will be in an unknown state — Core will report this. Resources

> already applied before the crash will have their state recorded correctly. You'd typically fix the issue (network connectivity, auth, provider bug) and re-run apply."

Q: Can you write your own provider?

> "Yes. Providers are Go programs that implement the Terraform Plugin Framework or Plugin SDK v2. HashiCorp has a provider development tutorial. In practice, platform teams sometimes write internal providers for things like provisioning from internal systems (internal DNS, internal CMDB, internal secret stores) that don't have public providers. For most use cases though, the 3000+ providers on the registry cover what you need."

🔵 Gotchas & Edge Cases

- ⚠ **Provider version and Terraform version compatibility** — not all provider versions work with all Terraform versions. The AWS provider 5.x requires Terraform 1.x. Always check compatibility when upgrading.
- **Provider schema is fetched fresh each time** — even with a cached binary, Core calls `GetProviderSchema` on every plan/apply. This is fast (local gRPC call) but worth knowing.
- **Providers are not terraform modules** — a common beginner confusion. Modules are reusable HCL configurations. Providers are Go binaries. Completely different things.
- **Data sources are provider-specific** — a data source from the AWS provider can only read AWS resources. You can't use `data.aws_*` to read GCP resources.
- **`null_resource` and `terraform_data`** — the `hashicorp/null` provider's `null_resource` is a special resource with no real infrastructure behind it, useful for running provisioners. In Terraform 1.4+, `terraform_data` (built into Core) replaces it.

🔵 Connections to Other Concepts

- → **Topic 9 (Versioning):** How to pin and control which provider version is used
- → **Topic 10 (Aliases):** Running multiple instances of the same provider
- → **Topic 11 (Caching):** How providers are stored and reused locally
- → **Category 6 (State):** Provider manages the mapping between HCL resources and real IDs in state
- → **Category 8 (Security):** Provider authentication is a critical security concern

---

Topic 8: Provider Configuration and Authentication Patterns

🔵 What It Is (Simple Terms)

Every provider needs to know **how to authenticate** with the API it manages. Provider configuration is where you tell the provider your credentials, target region, project, endpoint, and other connection settings.

Getting authentication right is critical — both for correctness (connecting to the right account/region) and security (not leaking credentials).

🔵 Why It Exists — What Problem It Solves

Without provider configuration, the provider doesn't know:

- **Which account** to manage (AWS account ID, GCP project, Azure subscription)
- **Which region** to operate in (us-east-1, eu-west-1)
- **Who you are** (access keys, service account, managed identity)
- **What endpoint** to hit (useful for custom endpoints, LocalStack, etc.)

The configuration also controls **per-provider settings** like retry behavior, request timeouts, and custom CA certificates.

🔵 The AWS Provider — Authentication Patterns

The AWS provider supports multiple authentication mechanisms, checked in this order:

```
┌──────────────────────────────────────────────────────────────────┐
│              AWS Provider Authentication Chain                   │
│                                                                  │
│  1. Static credentials in provider block (⚠ never in prod)       │
│  2. Environment variables (AWS_ACCESS_KEY_ID, etc.)              │
│  3. Shared credentials file (~/.aws/credentials)                 │
│  4. AWS profile (named profile in ~/.aws/config)                 │
│  5. Container credentials (ECS task role, EKS pod identity)      │
│  6. Instance metadata service (EC2 instance role, IMDSv2)        │
│  7. AssumeRole via STS                                           │
└──────────────────────────────────────────────────────────────────┘
```

Pattern 1: Static Credentials (⚠ Development Only — Never Production)

```
provider "aws" {
  region     = "us-east-1"
  access_key = "AKIAIOSFODNN7EXAMPLE"       # ⚠ NEVER hardcode
  secret_key = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"  # ⚠ NEVER
hardcode
}
```

> ⚠ **Never hardcode credentials in `.tf` files.** They end up in Git history and are a critical security vulnerability. This pattern exists for documentation purposes only.

Pattern 2: Environment Variables (CI/CD Standard)

```
# provider block — no credentials, reads from environment
provider "aws" {
  region = "us-east-1"
}
```

```
# Set in CI/CD pipeline environment or shell
export AWS_ACCESS_KEY_ID="AKIAIOSFODNN7EXAMPLE"
export AWS_SECRET_ACCESS_KEY="wJalrXUtnFEMI/..."
export AWS_SESSION_TOKEN="FQoDYXdz..."       # if using temporary
credentials
export AWS_DEFAULT_REGION="us-east-1"        # can also set region here
```

Pattern 3: AWS Profile (Local Development)

```
provider "aws" {
  region  = "us-east-1"
  profile = "mycompany-dev"    # references ~/.aws/credentials [mycompany-
dev]
}
```

```
# ~/.aws/credentials
[mycompany-dev]
aws_access_key_id     = AKIAIOSFODNN7EXAMPLE
aws_secret_access_key = wJalrXUtnFEMI/...

[mycompany-prod]
aws_access_key_id     = AKIAI...
aws_secret_access_key = xyz...
```

Pattern 4: IAM Role Assumption (Most Secure — Production Standard)

```
provider "aws" {
  region = "us-east-1"

  assume_role {
    role_arn     = "arn:aws:iam::123456789012:role/TerraformExecutionRole"
    session_name = "terraform-session"
    external_id  = "unique-external-id"      # for third-party access
  }
}
```

```
How AssumeRole works:
1. Terraform runs with base credentials (EC2 instance role, CI/CD OIDC
token, etc.)
2. Provider calls STS AssumeRole API with the target role ARN
3. STS returns temporary credentials (AccessKey, SecretKey, SessionToken)
4. Provider uses temporary credentials for all subsequent API calls
5. Temporary credentials expire (1 hour by default) — Terraform refreshes
automatically
```

Why AssumeRole is the production standard:

- No long-lived static credentials to manage or rotate
- Clear audit trail — every API call shows which role was assumed
- Can cross account boundaries — assume roles in other AWS accounts
- Works with least-privilege — the base identity needs only `sts:AssumeRole`

Pattern 5: OIDC / Workload Identity (Most Modern — CI/CD Best Practice)

```
# No credentials in provider block at all
provider "aws" {
  region = "us-east-1"
  # Credentials come from OIDC token exchanged for IAM role
  # Configured at CI/CD platform level, not in Terraform
}
```

```yaml
# GitHub Actions example
jobs:
  terraform:
    permissions:
      id-token: write     # Required for OIDC
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/GitHubActionsRole
          aws-region: us-east-1
      - run: terraform apply -auto-approve
```

```
OIDC Flow:
GitHub Actions → gets OIDC token from GitHub's token endpoint
                 → exchanges token with AWS STS via
AssumeRoleWithWebIdentity
                 → receives temporary AWS credentials
```

```
                    → credentials injected as environment variables
Terraform AWS provider → reads AWS_ACCESS_KEY_ID etc. from environment
```

Why OIDC is best practice:

- Zero static credentials — no secrets to store in CI/CD
- Credentials are scoped to the specific job/repo/branch
- Automatically rotated (short-lived tokens)
- Fully auditable via CloudTrail

🔵 GCP Provider Authentication Patterns

```
provider "google" {
  project = "my-gcp-project"
  region  = "us-central1"
  zone    = "us-central1-a"
}
```

```
# Pattern 1: Application Default Credentials (local dev)
gcloud auth application-default login

# Pattern 2: Service account key file (⚠ avoid in prod)
export GOOGLE_APPLICATION_CREDENTIALS="/path/to/service-account.json"

# Pattern 3: Workload Identity (GKE / Cloud Run) — best practice
# No credentials needed — GCP assigns identity to the workload
automatically
```

```
# Pattern 4: Impersonate service account
provider "google" {
  project                     = "my-project"
  impersonate_service_account = "terraform@my-
project.iam.gserviceaccount.com"
}
```

🔵 Azure Provider Authentication Patterns

```
provider "azurerm" {
  features {}                          # required even if empty
  subscription_id = "00000000-0000-0000-0000-000000000000"
}
```

```
# Pattern 1: Azure CLI (local dev)
az login

# Pattern 2: Service Principal with client secret
export ARM_CLIENT_ID="00000000-..."
export ARM_CLIENT_SECRET="..."
export ARM_TENANT_ID="00000000-..."
export ARM_SUBSCRIPTION_ID="00000000-..."

# Pattern 3: Managed Identity (Azure VMs, ACI, AKS) — best practice
# provider "azurerm" { use_msi = true }
```

🔵 Provider Configuration Best Practices

```
# ✅ Best practice: use variables for region, avoid hardcoding
provider "aws" {
  region = var.aws_region

  default_tags {                        # Applied to ALL resources
    tags = {
      ManagedBy   = "terraform"
      Environment = var.environment
      Repository  = "github.com/myorg/infra"
    }
  }
}
```

```
# ✅ Best practice: never put credentials in provider block
# Use environment variables, instance roles, or OIDC

# ✅ Best practice: use assume_role for cross-account access
provider "aws" {
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::${var.account_id}:role/TerraformRole"
  }
}
```

🔵 Short Interview Answer

> "Provider configuration tells the provider how to authenticate and which account/region to target. The most common patterns are environment variables for CI/CD, named profiles for local development, and IAM role assumption for production. The current best practice is OIDC-based authentication in CI/CD — no static credentials at all. The CI platform gets a short-lived OIDC token

> that it exchanges with AWS STS for temporary credentials. This eliminates secret management overhead entirely."

🔵 Real World Production Architecture

```
┌──────────────────────────────────────────────────────────────┐
│              Production Auth Architecture                    │
│                                                              │
│  GitHub Actions                                              │
│       │ OIDC token (JWT)                                     │
│       ▼                                                      │
│  AWS STS AssumeRoleWithWebIdentity                           │
│       │ temporary credentials (1hr)                          │
│       ▼                                                      │
│  TerraformExecutionRole (IAM Role)                           │
│  ├── AdministratorAccess (dev/staging)                       │
│  └── Custom policy (prod — least privilege)                  │
│       │                                                      │
│       ▼                                                      │
│  Terraform AWS Provider                                      │
│  └── Makes API calls as TerraformExecutionRole               │
└──────────────────────────────────────────────────────────────┘
```

🔵 Common Interview Questions

Q: How do you avoid hardcoding AWS credentials in Terraform?

> "Several ways, in order of preference: First, OIDC in CI/CD — GitHub Actions/GitLab CI exchange OIDC tokens for temporary AWS credentials, no secrets stored anywhere. Second, IAM instance roles — if Terraform runs on EC2 or EKS, the compute resource has an IAM role and credentials are fetched from the metadata service. Third, environment variables — credentials injected by the CI/CD system as environment variables, not stored in code. The AWS provider reads `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` automatically. Static credentials in the provider block should never be used."

Q: What is `assume_role` in the AWS provider and why use it?

> "`assume_role` tells the AWS provider to call STS AssumeRole before making any API calls. The provider uses your base credentials (whatever they are — env vars, instance role, OIDC) to assume a specific IAM role, then uses that role's temporary credentials for all Terraform operations. This is the standard pattern for multi-account setups — your CI has one base identity, and it assumes different roles in different accounts (dev, staging, prod) depending on what's being deployed."

Q: What is `default_tags` in the AWS provider?

> "`default_tags` is a provider-level configuration block that applies specified tags to every resource managed by that provider instance. Instead of copy-pasting `ManagedBy = terraform` and

> `Environment = prod` into every resource, you define them once in the provider. Resources can still override or add tags. This is a huge quality-of-life feature for consistent tagging compliance."

🔵 Gotchas & Edge Cases

- ⚠ **`default_tags` can cause unexpected diffs** — if an existing resource has tags that conflict with `default_tags`, Terraform will want to update them. Be careful when adding `default_tags` to existing configs.
- ⚠ **Region is not always `us-east-1`** — some global AWS resources (IAM, Route53, ACM for CloudFront) must use `us-east-1` regardless. Use provider aliases for this.
- **Environment variables take precedence over profile** — if `AWS_ACCESS_KEY_ID` is set in the environment, it overrides the `profile` setting in the provider block.
- **Session token is required with temporary credentials** — if using `assume_role` output or temporary credentials, you must set `AWS_SESSION_TOKEN` alongside key/secret, or the provider will get `InvalidClientTokenId` errors.
- **GCP `GOOGLE_APPLICATION_CREDENTIALS` vs `GOOGLE_CREDENTIALS`** — the env var differs by provider version; always check the provider docs.
- **Azure `features {}` block is mandatory** — even if empty, omitting it causes an error. This surprises people coming from other providers.

🔵 Connections to Other Concepts

- → **Topic 10 (Aliases):** Multiple provider configs for cross-region/account use the same auth patterns
- → **Category 8 (Security):** OIDC, least privilege, and credential management are security-critical
- → **Category 9 (CI/CD):** OIDC authentication is the modern CI/CD standard
- → **Category 10 (Multi-account patterns):** AssumeRole is how multi-account Terraform works

---

Topic 9: Provider Versioning and `required_providers`

🔵 What It Is (Simple Terms)

`required_providers` is the block where you declare **which providers your configuration needs**, where to download them from, and **what versions are acceptable**. It's your dependency manifest — like `dependencies` in `package.json`.

🔵 Why It Exists — What Problem It Solves

Without `required_providers`:

- Terraform downloads the **latest** version of any provider it finds referenced
- Different engineers get different provider versions

- CI might use a different version than production
- A provider update with breaking changes silently breaks your config
- There's no record of which provider was intended

`required_providers` + `.terraform.lock.hcl` together give you **deterministic, reproducible provider resolution**.

🔵 The `required_providers` Block

```
terraform {
  required_version = ">= 1.5.0"     # Terraform Core version constraint

  required_providers {
    aws = {
      source  = "hashicorp/aws"      # <namespace>/<type> on
registry.terraform.io
      version = "~> 5.0"             # version constraint
    }

    google = {
      source  = "hashicorp/google"
      version = ">= 4.0, < 6.0"     # multiple constraints
    }

    datadog = {
      source  = "datadog/datadog"    # non-HashiCorp namespace
      version = "~> 3.27"
    }

    # Custom/internal provider
    mycompany = {
      source  = "registry.mycompany.com/internal/myplatform"  # private
registry
      version = "~> 1.0"
    }
  }
}
```

🔵 Version Constraint Operators

Operator	Meaning	Example	Matches
`=`	Exact version	`= 5.1.0`	Only 5.1.0
`!=`	Not this version	`!= 5.1.0`	Anything except 5.1.0
`>`	Greater than	`> 5.0`	5.1, 5.2, 6.0...
`>=`	Greater than or equal	`>= 5.0`	5.0, 5.1, 6.0...

Operator	Meaning	Example	Matches
`<`	Less than	`< 6.0`	5.x, 4.x...
`<=`	Less than or equal	`<= 5.9`	5.9, 5.8, 5.0...
`~>`	Pessimistic constraint (most important)	`~> 5.0`	>= 5.0, < 6.0
`~>`	Pessimistic (patch)	`~> 5.1.0`	>= 5.1.0, < 5.2.0

The Pessimistic Constraint Operator `~>` Explained

```
~> 5.0   means: "allow rightmost version segment to increment"
         = >= 5.0, < 6.0
         Allows: 5.0, 5.1, 5.99
         Blocks: 6.0 (potential breaking changes)

~> 5.1.0 means: "allow only patch versions to increment"
         = >= 5.1.0, < 5.2.0
         Allows: 5.1.0, 5.1.1, 5.1.99
         Blocks: 5.2.0 (potential minor breaking changes)
```

Which to use in practice:

```
# In application modules (used by others) — be liberal, don't over-
constrain
version = ">= 5.0, < 6.0"     # Accept any 5.x

# In root modules (your own infra) — be specific, lock to minor version
version = "~> 5.1"             # Allow 5.1.x through 5.x but not 6.x

# In production infra — be very specific after testing
version = "~> 5.1.0"           # Lock to patch version only
```

🔵 Version Constraint Strategy by Module Type

```
Root Module (your live infrastructure)
├── Use ~> with minor version: ~> 5.1
├── Let lock file pin the exact patch version
└── Upgrade deliberately with terraform init -upgrade

Reusable Module (shared module used by others)
├── Use >= with upper bound: >= 5.0, < 6.0
├── Don't pin exact versions — consumers need flexibility
└── Document tested provider versions in README

Library Module (internal platform module)
```

```
├── Match your organization's standard versions
└── Coordinate upgrades across all consuming root modules
```

🔵 The Full Version Resolution Flow

```
1. Read required_providers version constraints from .tf files
       │
       ▼
2. Check .terraform.lock.hcl for locked version
   ├── Lock file has compatible version → use locked version
   └── Lock file missing or incompatible → resolve constraint
              │
              ▼
3. Query registry for available versions
       │
       ▼
4. Select latest version satisfying ALL constraints
       │
       ▼
5. Download provider binary for current OS/arch
       │
       ▼
6. Verify checksum against lock file (or write new lock entry)
       │
       ▼
7. Store in .terraform/providers/
```

🔵 `required_version` for Terraform Core

```
terraform {
  required_version = ">= 1.5.0, < 2.0.0"
}
```

This constrains which **Terraform Core version** can run this config. If someone tries to run with Terraform 0.14, they get an error immediately. This is important because HCL features, behavior, and state format differ across Terraform versions.

```
# Check current Terraform version
terraform version

# Error you'll see if version doesn't match:
# Error: Unsatisfied Terraform version constraint
# The local Terraform version is 1.4.6, but this config requires >= 1.5.0
```

🔵 Upgrading Providers

```
# See current provider versions
terraform version                    # shows Terraform + provider versions
cat .terraform.lock.hcl              # shows locked versions

# Upgrade all providers to latest within constraints
terraform init -upgrade

# What -upgrade does:
# 1. Ignores current lock file
# 2. Resolves constraints fresh (picks latest satisfying versions)
# 3. Updates .terraform.lock.hcl with new versions + checksums

# After upgrade, ALWAYS review the diff:
git diff .terraform.lock.hcl         # What changed?
terraform plan                       # Any unexpected resource changes?
```

🔵 Short Interview Answer

> "`required_providers` declares which providers a Terraform configuration needs, their source addresses on the registry, and acceptable version ranges. The most common constraint operator is `~>` (pessimistic) which allows minor/patch updates but blocks major version changes. In root modules you'd use `~> 5.1` to allow patch updates. In reusable modules you'd use `>= 5.0, < 6.0` to give consumers flexibility. The exact version is pinned by `.terraform.lock.hcl` after `init`, and upgraded deliberately with `terraform init -upgrade`."

🔵 Real World Production Example

```
# Root module version pinning for a production environment
terraform {
  required_version = ">= 1.6.0, < 2.0.0"

  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.31"     # tested with 5.31.x — allows patch updates
    }
    kubernetes = {
      source  = "hashicorp/kubernetes"
      version = "~> 2.24"
    }
    helm = {
      source  = "hashicorp/helm"
      version = "~> 2.12"
    }
```

```
    datadog = {
      source  = "datadog/datadog"
      version = "~> 3.27"
    }
  }
}

# Reusable VPC module — more permissive to work with different root
modules
# In module/vpc/versions.tf:
terraform {
  required_version = ">= 1.3.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = ">= 4.0, < 6.0"   # works with AWS provider 4.x or 5.x
    }
  }
}
```

🔵 Common Interview Questions

Q: What is the difference between `~> 5.0` and `~> 5.1.0`?

> "`~> 5.0` allows any 5.x version — 5.0, 5.1, 5.99 — but blocks 6.0. It's equivalent to `>= 5.0, < 6.0`. `~> 5.1.0` is more restrictive — it allows 5.1.x patches but blocks 5.2.0. It's equivalent to `>= 5.1.0, < 5.2.0`. The rule is: the pessimistic operator allows the rightmost specified segment to increment. Use `~> 5.1.0` in root modules when you want tighter control, and `~> 5.0` when you want to accept all minor updates within a major version."

Q: Why should reusable modules use looser version constraints?

> "Because a reusable module is consumed by many root modules, each potentially using different provider versions. If your module requires exactly `~> 5.1.0` and the consumer is already using `5.2.0`, there's a conflict. Modules should use the minimum version that includes needed features and an upper bound on the major version — like `>= 5.0, < 6.0`. The root module's lock file determines the actual resolved version. This is the same principle as library versioning in software packages."

Q: What does `terraform init -upgrade` do?

> "It re-resolves all provider version constraints ignoring the current lock file, downloads the latest versions satisfying your constraints, and updates `.terraform.lock.hcl`. You should always run `terraform plan` after an upgrade to check for any unexpected changes caused by the new provider version. Provider upgrades can sometimes cause no-op plans to show changes (especially with attribute handling changes), so reviewing the lock file diff in code review is important."

🔵 Gotchas & Edge Cases

- ⚠ **No `required_providers` = Terraform guesses the source** — before Terraform 0.13, you could use providers without declaring them. In modern Terraform, not declaring a provider won't cause an immediate error but Terraform will try to guess the source as `hashicorp/<type>` which may not exist.
- ⚠ **Multiple modules with conflicting constraints cause errors** — if root module requires `~> 5.0` and a child module requires `>= 5.1`, Terraform resolves to the most restrictive. But if constraints are mutually exclusive (`= 5.0.0` and `= 5.1.0`), you get a "no available versions" error.
- **`required_version` is checked before everything else** — if the Terraform binary doesn't match, you get an error before any providers are initialized.
- **Provider source is case-sensitive in the registry** — `hashicorp/AWS` is not the same as `hashicorp/aws`.
- **Removing a provider from `required_providers` doesn't remove it from state** — resources managed by that provider still exist in state and will cause errors on next plan.

🔵 Connections to Other Concepts

- → **Topic 6 (Lock File):** `required_providers` sets constraints; lock file records the resolution
- → **Topic 10 (Aliases):** Multiple instances of the same provider all use the same versioned binary
- → **Category 7 (Modules):** Module version constraints must be compatible with root module constraints
- → **Category 9 (CI/CD):** Provider upgrades are managed through CI/CD pipeline runs

---

Topic 10: ⚠ Multiple Provider Instances — Aliases and Cross-Region/Account

---

🔵 What It Is (Simple Terms)

By default, you can only have one configuration of each provider per Terraform module. But in the real world, you often need to manage resources in **multiple AWS regions** (e.g., create an ACM certificate in `us-east-1` for CloudFront while your main infra is in `eu-west-1`) or **multiple AWS accounts** (dev account and prod account in the same config).

Provider aliases solve this — they let you define multiple named instances of the same provider with different configurations.

> ⚠ This is one of the most commonly tested intermediate-to-advanced Terraform topics. Interviewers probe whether you understand both the syntax AND the architecture of why aliases are needed.

🔵 Why It Exists — What Problem It Solves

Problem 1: Multi-region resources

```
CloudFront distribution requires ACM certificate in us-east-1
Your infrastructure lives in eu-west-1
You can't use the default provider (eu-west-1) to create the ACM cert
```

Problem 2: Multi-account deployments

```
Platform team manages shared networking (VPC) in account A
Application team deploys into account B
A single Terraform config needs to create resources in both accounts
```

Problem 3: Provider configuration differences

```
Some resources need different timeouts, endpoints, or settings
A secondary provider instance with different config handles those
resources
```

🔵 Defining Provider Aliases

```
# Default provider (no alias) — used when no provider is specified
provider "aws" {
  region = "eu-west-1"
}

# Aliased provider — must be explicitly referenced
provider "aws" {
  alias  = "us_east_1"             # alias name — used to reference this
instance
  region = "us-east-1"
}

# Another aliased provider — different account via assume_role
provider "aws" {
  alias  = "prod_account"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::999999999999:role/TerraformRole"
  }
}
```

Rules:

- One provider block **without** `alias` = the default provider
- All other instances **must** have `alias`
- Resources use the default provider unless you specify `provider = aws.<alias>`

🔵 Using Aliased Providers in Resources

```
# Default provider (eu-west-1) — no provider argument needed
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

# Aliased provider — must specify provider argument
resource "aws_acm_certificate" "cloudfront" {
  provider          = aws.us_east_1     # syntax: <provider_type>.<alias>
  domain_name       = "example.com"
  validation_method = "DNS"
}

# Using the prod account provider
resource "aws_s3_bucket" "prod_logs" {
  provider = aws.prod_account
  bucket   = "prod-application-logs"
}
```

🔵 Real World: CloudFront + ACM Pattern

This is the most common real-world alias pattern — every CloudFront distribution requires its ACM cert in `us-east-1`.

```
# providers.tf
provider "aws" {
  region = var.aws_region             # e.g. eu-west-1 — default for all
resources
}

provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"               # required for CloudFront ACM certs
}

# acm.tf — certificate MUST be in us-east-1 for CloudFront
resource "aws_acm_certificate" "cdn" {
  provider          = aws.us_east_1   # explicitly use the us-east-1
provider
  domain_name       = "cdn.example.com"
  validation_method = "DNS"
}

# cloudfront.tf — CloudFront is global, uses default provider
resource "aws_cloudfront_distribution" "cdn" {
  # uses default provider
  viewer_certificate {
```

```
    acm_certificate_arn = aws_acm_certificate.cdn.arn   # reference us-
east-1 cert
  }
}
```

🔵 Real World: Multi-Account Pattern

```
# providers.tf

# CI/CD role in the tools account — base identity
provider "aws" {
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::111111111111:role/TerraformRole"  # tools
account
  }
}

# Dev account
provider "aws" {
  alias  = "dev"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/TerraformRole"  # dev
account
  }
}

# Prod account
provider "aws" {
  alias  = "prod"
  region = "eu-west-1"
  assume_role {
    role_arn = "arn:aws:iam::333333333333:role/TerraformRole"  # prod
account
  }
}

# main.tf
resource "aws_vpc" "dev" {
  provider   = aws.dev
  cidr_block = "10.1.0.0/16"
}

resource "aws_vpc" "prod" {
  provider   = aws.prod
  cidr_block = "10.0.0.0/16"
}
```

🔵 Provider Aliases in Modules

Passing provider aliases into modules requires **explicit provider passing** — modules do not inherit aliases automatically.

```
# Root module — define the providers
provider "aws" {
  region = "eu-west-1"
}

provider "aws" {
  alias  = "us_east_1"
  region = "us-east-1"
}

# Call a module that needs both providers
module "cdn" {
  source = "./modules/cdn"

  # Explicit provider mapping: module_provider_name = root_module_provider
  providers = {
    aws           = aws               # default provider → module's
default aws
    aws.us_east_1 = aws.us_east_1    # aliased provider → module's aliased
aws
  }
}
```

```
# modules/cdn/providers.tf — module declares the providers it expects
terraform {
  required_providers {
    aws = {
      source                = "hashicorp/aws"
      version               = "~> 5.0"
      configuration_aliases = [aws.us_east_1]    # declare alias
requirement
    }
  }
}

# modules/cdn/main.tf
resource "aws_cloudfront_distribution" "this" {
  # uses default aws provider
}

resource "aws_acm_certificate" "this" {
  provider = aws.us_east_1    # uses the passed-in aliased provider
}
```

🔵 Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────────┐
│                  Multi-Provider Architecture                         │
│                                                                      │
│  Root Module                                                         │
│  ┌──────────────────────────────────────────────────────────────┐    │
│  │  provider "aws" { region = "eu-west-1" }   ← default         │    │
│  │  provider "aws" { alias = "us_east" ... }  ← aliased         │    │
│  │  provider "aws" { alias = "prod_acct" ...} ← aliased         │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                │ providers = { ... }                 │
│  ┌─────────────────────────────▼────────────────────────────────┐    │
│  │  module "cdn"                                                │    │
│  │  ├── aws_cloudfront_distribution (default provider)          │    │
│  │  └── aws_acm_certificate (aws.us_east provider)              │    │
│  └──────────────────────────────────────────────────────────────┘    │
│                                                                      │
│  Provider Processes (separate OS processes):                         │
│  ├── terraform-provider-aws (eu-west-1, tools account)               │
│  ├── terraform-provider-aws (us-east-1, tools account)               │
│  └── terraform-provider-aws (eu-west-1, prod account)                │
└──────────────────────────────────────────────────────────────────────┘
```

🔵 Short Interview Answer

> "Provider aliases let you define multiple configurations of the same provider — different regions, different accounts, different credentials. You define a provider block with an `alias` parameter, then reference it in resources using `provider = aws.<alias>`. The most common real-world use case is CloudFront requiring ACM certificates in `us-east-1` while your infrastructure is in a different region. For modules, you must explicitly pass aliased providers using the `providers` map argument — modules don't inherit aliases automatically."

🔵 Deep Dive Answer

> "When you define multiple provider instances with aliases, Terraform Core starts a separate process for each provider instance. They're the same binary but configured differently — different AWS SDK clients pointing to different regions or using different STS role credentials. Resources explicitly reference which instance to use via the `provider` meta-argument. In modules, the `configuration_aliases` in `required_providers` declares which aliases the module expects, and the root module maps its providers to the module's expectations via the `providers` argument. A common gotcha is forgetting to pass aliases into modules — the module will silently use the default provider, which may be pointing at the wrong region or account."

🔵 Common Interview Questions

Q: When would you use provider aliases?

> "Three main scenarios: First, multi-region — when some resources must be in a specific region regardless of where your main infra lives (ACM for CloudFront must be us-east-1). Second, multi-account — a single Terraform config managing resources across multiple AWS accounts, using AssumeRole with different role ARNs per provider instance. Third, different provider configurations for different resource groups — like different timeouts, custom endpoints for testing, or different credentials for different resource types."

Q: How do you pass a provider alias into a module?

> "Two parts. In the module, declare the alias requirement in `required_providers` using `configuration_aliases = [aws.us_east_1]`. In the calling root module, pass the provider mapping using the `providers` argument: `providers = { aws = aws, aws.us_east_1 = aws.us_east_1 }`. If you don't declare `configuration_aliases` in the module, the module can't use the aliased provider even if it's passed in."

Q: What happens if you don't define a default (non-aliased) provider?

> "If all your provider blocks have aliases and none is the default, any resource without an explicit `provider` argument will fail with an error — Terraform can't find a default provider configuration. You always need one non-aliased provider block as the default, or you must explicitly specify `provider = aws.<alias>` on every single resource, which is impractical."

🔵 Gotchas & Edge Cases

- ⚠ **Modules don't inherit aliases** — this is the #1 mistake. If you define `aws.us_east_1` in the root module but don't explicitly pass it to a module via `providers = {}`, the module uses only the default provider.
- ⚠ **`configuration_aliases` must match exactly** — the alias name in the module's `configuration_aliases` must match the alias name passed in the `providers` map.
- **Each alias = a separate provider process** — with 10 aliased providers, you have 10 provider subprocesses running. This has minor performance implications.
- **`for_each` on provider aliases is not supported** — you can't dynamically create provider instances with `for_each`. If you need 50 region providers, you must define 50 provider blocks. (Terraform 1.7+ is working on this limitation.)
- **`terraform plan -target` with aliases** — targeting a resource that uses an aliased provider works, but be careful — the aliased provider still initializes and connects.
- **Provider aliases in state** — the state file records which provider alias manages each resource. If you rename an alias, Terraform will think resources need to be recreated.

🔵 Connections to Other Concepts

- → **Topic 8 (Auth):** Each aliased provider has its own auth config — critical for multi-account
- → **Category 7 (Modules):** Module provider passing is tightly coupled to aliases
- → **Category 10 (Multi-account patterns):** Aliases are the mechanism for multi-account Terraform
- → **Category 5 (Meta-arguments):** `provider` is a meta-argument available on all resources

---

Topic 11: Provider Caching and Plugin Mirror Strategies

🔵 What It Is (Simple Terms)

Every time you run `terraform init` in a new directory or on a fresh machine, Terraform downloads provider binaries from `registry.terraform.io`. This is slow (100MB+ for the AWS provider), bandwidth-heavy, and impossible in air-gapped (no-internet) environments.

Provider caching and **plugin mirrors** solve this by storing provider binaries locally or on an internal server so they don't need to be re-downloaded repeatedly.

🔵 Why It Exists — What Problem It Solves

Scenario	Problem	Solution
Large team, many workspaces	Each `init` downloads 100MB+ per provider	Filesystem cache
CI/CD pipelines	Every pipeline run re-downloads all providers	CI cache or mirror
Air-gapped environments	No internet access to registry.terraform.io	Network mirror
Corporate security policy	All downloads must go through approved channels	Implicit mirror (Artifactory, Nexus)
Slow registry response	registry.terraform.io is slow or unavailable	Local filesystem mirror

🔵 Option 1: Plugin Cache Directory (Local Caching)

The simplest form of caching — a shared local directory where downloaded providers are cached.

```
# ~/.terraformrc (global Terraform config file on Linux/Mac)
# or %APPDATA%\terraform.rc on Windows

plugin_cache_dir = "$HOME/.terraform.d/plugin-cache"
```

```
# Or set via environment variable
export TF_PLUGIN_CACHE_DIR="$HOME/.terraform.d/plugin-cache"
```

How it works:

```
First terraform init in any directory:
  → Downloads provider to .terraform/providers/ (project-local)
  → ALSO copies to $TF_PLUGIN_CACHE_DIR

Second terraform init in same or different directory:
  → Finds provider in cache directory
  → Creates symlink from .terraform/providers/ → cache directory
  → No download needed
```

Directory structure:

```
~/.terraform.d/plugin-cache/
└── registry.terraform.io/
    └── hashicorp/
        └── aws/
            └── 5.1.0/
                └── linux_amd64/
                    └── terraform-provider-aws_v5.1.0_x5   ← binary cached
here
```

Important: The cache directory only works for exact version+OS+arch combinations. A new version still downloads.

🔵 Option 2: Filesystem Mirror

A filesystem mirror is a local directory that mirrors the provider registry structure. Unlike the cache, it is **explicitly served as the source** — not just a cache.

```
# ~/.terraformrc
provider_installation {
  filesystem_mirror {
    path    = "/opt/terraform/providers"
    include = ["registry.terraform.io/*/*"]   # mirror all providers
  }

  # Fall back to direct for anything not in the mirror
  direct {
    exclude = ["registry.terraform.io/*/*"]   # don't go to registry if in
mirror
  }
}
```

Populating a filesystem mirror:

```
# Download providers to the mirror directory
terraform providers mirror /opt/terraform/providers

# This creates the mirror structure:
/opt/terraform/providers/
└── registry.terraform.io/
    └── hashicorp/
        └── aws/
            └── terraform-provider-aws_5.1.0_linux_amd64.zip
            └── terraform-provider-aws_5.1.0_darwin_arm64.zip
            └── terraform-provider-aws_5.1.0_SHA256SUMS
            └── terraform-provider-aws_5.1.0_SHA256SUMS.sig
```

🔵 Option 3: Network Mirror (Enterprise/Air-Gapped)

A network mirror is an HTTPS server that implements the Terraform Provider Mirror Protocol. Tools like Artifactory, Nexus Repository, and JFrog can serve as Terraform provider mirrors.

```
# ~/.terraformrc
provider_installation {
  network_mirror {
    url     = "https://terraform-mirror.mycompany.com/providers/"
    include = ["registry.terraform.io/hashicorp/*"]   # only mirror
HashiCorp providers
  }

  direct {
    include = ["registry.terraform.io/mycompany/*"]   # direct for
internal providers
  }
}
```

How network mirrors work:

```
terraform init
  → Checks .terraformrc for provider_installation config
  → Requests provider index from mirror URL:
    GET https://terraform-mirror.mycompany.com/providers/
        registry.terraform.io/hashicorp/aws/index.json
  → Gets available versions list
  → Downloads provider zip from mirror (not registry)
  → Verifies checksums
```

🔵 Option 4: Terraform Cloud / Enterprise as Mirror

Terraform Cloud and Enterprise have a built-in **private provider registry** that can also act as a mirror for public providers. This is common in enterprises.

```
terraform {
  cloud {
    organization = "mycompany"
    workspaces { name = "prod-infra" }
  }
}

# TFC automatically uses its provider registry
# No explicit mirror config needed when running in TFC
```

🔵 The `provider_installation` Block in Full

```
# ~/.terraformrc — full example with priority order

plugin_cache_dir = "$HOME/.terraform.d/plugin-cache"

provider_installation {
  # 1st priority: internal network mirror (all providers)
  network_mirror {
    url     = "https://nexus.mycompany.com/terraform/"
    include = ["registry.terraform.io/hashicorp/*",
               "registry.terraform.io/datadog/*"]
  }

  # 2nd priority: local filesystem mirror (for air-gapped fallback)
  filesystem_mirror {
    path    = "/mnt/shared/terraform-providers"
    include = ["registry.terraform.io/hashicorp/*"]
  }

  # 3rd priority: direct from registry (everything else)
  direct {
    exclude = ["registry.terraform.io/hashicorp/*",
               "registry.terraform.io/datadog/*"]
  }
}
```

🔵 CI/CD Caching Strategy

```
# GitHub Actions — caching providers between runs
jobs:
  terraform:
```

```yaml
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      # Cache Terraform providers
      - uses: actions/cache@v3
        with:
          path: ~/.terraform.d/plugin-cache
          key: ${{ runner.os }}-terraform-${{
hashFiles('**/.terraform.lock.hcl') }}
          restore-keys: |
            ${{ runner.os }}-terraform-

      - name: Configure cache directory
        run: |
          mkdir -p ~/.terraform.d/plugin-cache
          echo 'plugin_cache_dir = "$HOME/.terraform.d/plugin-cache"' >
~/.terraformrc

      - name: Terraform Init
        run: terraform init
        # First run: downloads providers, saves to cache
        # Subsequent runs with same lock file: uses cache, no download
```

🔵 `terraform providers` Commands

```bash
# List all providers required by current config
terraform providers

# Output:
# Providers required by configuration:
# .
# └── provider[registry.terraform.io/hashicorp/aws] ~> 5.0

# Download providers to a mirror directory
terraform providers mirror ./mirror-dir

# Lock providers for multiple platforms (updates lock file)
terraform providers lock \
  -platform=linux_amd64 \
  -platform=darwin_arm64 \
  -platform=windows_amd64
```

🔵 Short Interview Answer

> "Provider caching stores downloaded provider binaries locally so `terraform init` doesn't re-download them every time. The simplest form is setting `plugin_cache_dir` in `.terraformrc` or `TF_PLUGIN_CACHE_DIR` environment variable — Terraform caches providers there and symlinks

> into each project's `.terraform` directory. For enterprise environments without internet access, you set up a network mirror using a tool like Artifactory that implements the Terraform mirror protocol, then configure `provider_installation` in `.terraformrc` to point there. In CI/CD, you cache the plugin cache directory between runs keyed on the lock file hash."

🔵 Real World Production Example

```
# Air-gapped setup script — run once on an internet-connected machine

# 1. Create mirror directory
mkdir -p /mnt/terraform-mirror

# 2. Download all required providers to mirror
cd /path/to/terraform/configs
terraform providers mirror /mnt/terraform-mirror

# 3. Transfer mirror directory to air-gapped network
rsync -av /mnt/terraform-mirror \
  airgapped-server:/opt/terraform/providers

# 4. Configure .terraformrc on air-gapped machines
cat > ~/.terraformrc << EOF
provider_installation {
  filesystem_mirror {
    path    = "/opt/terraform/providers"
    include = ["registry.terraform.io/*/*"]
  }
  direct {
    exclude = ["registry.terraform.io/*/*"]
  }
}
EOF

# 5. Now terraform init works without internet
terraform init
# ✓ Installed hashicorp/aws v5.1.0 (unauthenticated)
```

🔵 Common Interview Questions

Q: How do you speed up `terraform init` in CI/CD?

> "Cache the provider binaries between runs. Set `TF_PLUGIN_CACHE_DIR` to a directory, configure your CI system to cache that directory, and key the cache on the `.terraform.lock.hcl` hash. When the lock file doesn't change (no provider updates), the cache hit means `init` runs in seconds instead of downloading hundreds of megabytes. When the lock file changes, the cache misses and new providers are downloaded and cached."

Q: How do you run Terraform in an air-gapped environment?

> "You need a provider mirror. On an internet-connected machine, run `terraform providers mirror <dir>` to download all required providers into a mirror directory structure. Transfer that directory to the air-gapped environment. Configure `.terraformrc` with a `provider_installation` block pointing to a `filesystem_mirror` with the path to your mirror directory, and set `direct { exclude = [...] }` to prevent attempts to reach the internet. For ongoing updates, you repeat the mirror process and re-sync."

Q: What is the difference between the plugin cache and a filesystem mirror?

> "The plugin cache (`plugin_cache_dir`) is a transparent local cache — Terraform still validates against the registry and uses it as a download shortcut. It's a performance optimization. A filesystem mirror is a full replacement for the registry — Terraform never contacts the public registry, it only looks in the mirror. The mirror must contain all required providers in the correct directory structure with proper checksums. Mirrors are used for air-gapped environments or strict security policies, while the cache is just for performance."

🔵 Gotchas & Edge Cases

- ⚠ **Plugin cache is not used for `terraform providers mirror`** — the mirror command always downloads fresh, it doesn't use the cache.
- ⚠ **Concurrent `terraform init` with shared cache can cause race conditions** — Terraform 1.0+ has file locking for the cache but older versions can corrupt the cache with parallel inits.
- **Cache doesn't work across major OS/arch changes** — a cache populated on `linux_amd64` won't serve `darwin_arm64`. Multi-platform CI needs careful cache key design.
- **Network mirrors must implement the full mirror protocol** — a simple nginx static file server won't work. The mirror must respond to the JSON index endpoints correctly. Artifactory, Nexus, and Cloudsmith all support this natively.
- **`TF_PLUGIN_CACHE_DIR` vs `.terraformrc`** — the environment variable takes precedence. Useful for CI where you don't want to create a `.terraformrc` file.
- **Checksum verification with mirrors** — Terraform still verifies checksums even with mirrors. If the lock file has checksums from the public registry and you're serving from a mirror, the checksums must match. This is why `terraform providers mirror` downloads the `SHA256SUMS` files too.

🔵 Connections to Other Concepts

- → **Topic 6 (Lock File):** The lock file's checksums are verified against cached/mirrored providers
- → **Topic 9 (Versioning):** Mirrors must contain the exact versions specified in version constraints
- → **Category 8 (Security):** Mirrors are a security control — all providers vetted before entering the mirror
- → **Category 9 (CI/CD):** Caching strategy directly impacts CI pipeline performance

---

📊 Category 2 Summary — Quick Reference Card

Topic	One-Line Summary	Interview Weight

Topic	One-Line Summary	Interview Weight
7. What Providers Are	Plugins, gRPC, separate process, translate to API calls	⭐⭐⭐⭐
8. Auth Patterns	Env vars, profiles, AssumeRole, OIDC — never hardcode	⭐⭐⭐⭐⭐
9. Versioning	`~>` operator, `required_providers`, lock file resolution	⭐⭐⭐⭐
10. Aliases ⚠	Multi-region/account, `providers = {}` in modules	⭐⭐⭐⭐⭐
11. Caching/Mirrors	`plugin_cache_dir`, filesystem mirror, air-gapped setup	⭐⭐⭐

🔑 Category 2 — Authentication Decision Tree

```
Where is Terraform running?
│
├── Local developer machine
│   └── Use AWS profile (~/.aws/credentials named profile)
│
├── CI/CD (GitHub Actions, GitLab CI)
│   └── Use OIDC (no static credentials)
│       └── Platform provides short-lived token →
AssumeRoleWithWebIdentity
│
├── EC2 instance / ECS task
│   └── Use instance/task IAM role (metadata service)
│       └── No credentials needed in provider config
│
├── Multi-account?
│   └── Use assume_role in provider block
│       └── Base identity assumes role in target account
│
└── Cross-region resource?
    └── Use provider alias
        └── provider "aws" { alias = "us_east_1", region = "us-east-1" }
```

🎯 Category 2 — Top 5 Interview Questions to Master

1. **"How does a Terraform provider work internally?"** — gRPC, subprocess, plugin protocol, CRUD RPC methods
2. **"How do you avoid hardcoding AWS credentials in Terraform?"** — OIDC, instance roles, env vars, AssumeRole
3. **"What does `~> 5.1` mean as a version constraint?"** — pessimistic operator, >= 5.1 < 6.0
4. **"How do you manage resources in multiple AWS regions in one Terraform config?"** — provider aliases, `configuration_aliases`, passing providers to modules
5. **"How would you run Terraform in an air-gapped environment?"** — filesystem mirror, `terraform providers mirror`, `.terraformrc` configuration

> **Next:** Category 3 — Resources, Data Sources & Dependencies (Topics 12–17) Type `Category 3` to continue, `quiz me` to be tested on Category 2, or `deeper` on any specific topic.